

EKS Assignment

Step 1: Clone the git repo to the local machine.

```
saksham@Cker-saksham:~$ git clone https://github.com/docker/getting-started-app.git
Cloning into 'getting-started-app'...
remote: Enumerating objects: 79, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 79 (delta 18), reused 15 (delta 15), pack-reused 51 (from 1)
Receiving objects: 100% (79/79), 1.67 MiB | 731.00 KiB/s, done.
Resolving deltas: 100% (19/19), done.
saksham@Cker-saksham:~$ ls
abc2.txt      Downloads      mysql-apr-config_0.8.29-1_all.deb.3  sample.war
abc3.txt      exercise       mysql-apr-config_0.8.29-1_all.deb.4  skey1
```

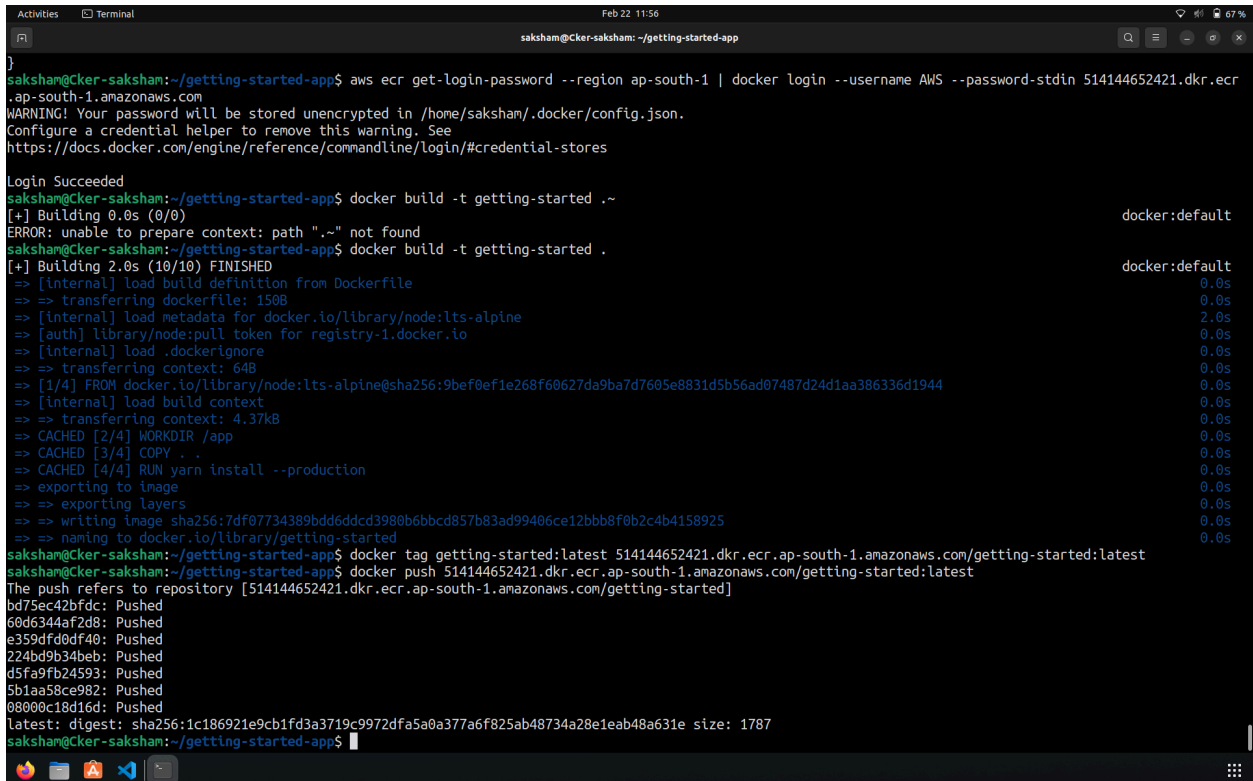
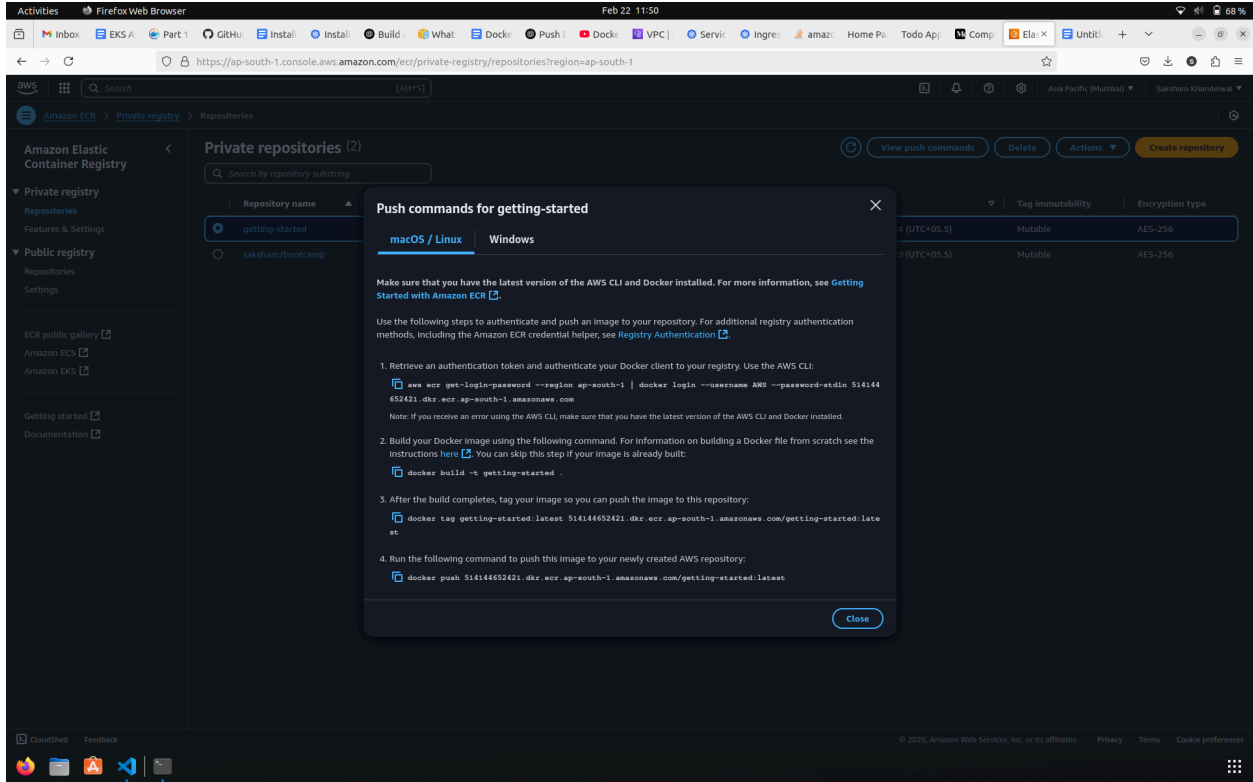
Step 2: Make a docker file for the application containerization.

```
EXPLORER
GETTING-STARTED-APP
  spec
  persistence
  .js sqlite.spec.js
  routes
  > src
  .dockerignore
  Dockerfile
Dockerfile
1 FROM node:lts-alpine
2 WORKDIR /app
3 COPY . .
4 RUN yarn install --production
5 CMD ["node", "src/index.js"]
6 EXPOSE 3000
```

Step 3: Make an ECR repo.

```
saksham@Cker-saksham:~/getting-started-app$ aws ecr create-repository --repository-name getting-started
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:514144652421:repository/getting-started",
    "registryId": "514144652421",
    "repositoryName": "getting-started",
    "repositoryUri": "514144652421.dkr.ecr.ap-south-1.amazonaws.com/getting-started",
    "createdAt": "2025-02-22T11:44:44.763000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

Step 4: Push the image to the ECR repo.



Step 5: Create an EKS cluster.

Activities Firefox Web Browser Feb 22, 12:06

https://ap-south-1.console.aws.amazon.com/eks/home?region=ap-south-1#/cluster-create

Search [Alt+S]

EKS > Clusters > Create EKS cluster

Step 1: **Configure cluster**

- Step 2: Specify networking
- Step 3: Configure observability
- Step 4: Select add-ons
- Step 5: Configure selected add-ons settings
- Step 6: Review and create

Configure cluster

Configuration options - new [Info](#)

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode) - new
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode - new [Info](#)

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode
EKS automates routing cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration [Info](#)

Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

Getting-Started-EKS

The cluster name must begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role [Info](#)
Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#).

EKS-Cluster1-Role [Create recommended role](#)

Kubernetes version settings

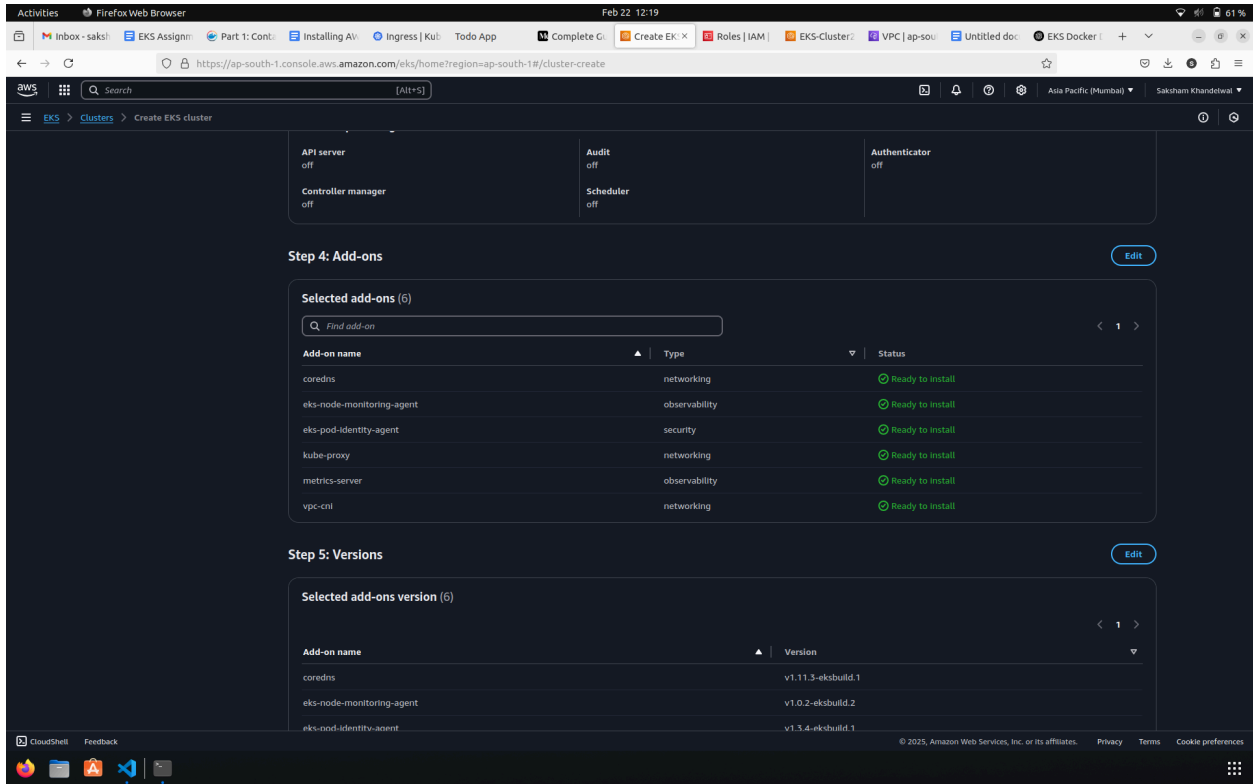
Kubernetes version [Info](#)
Select Kubernetes version for this cluster.

1.31

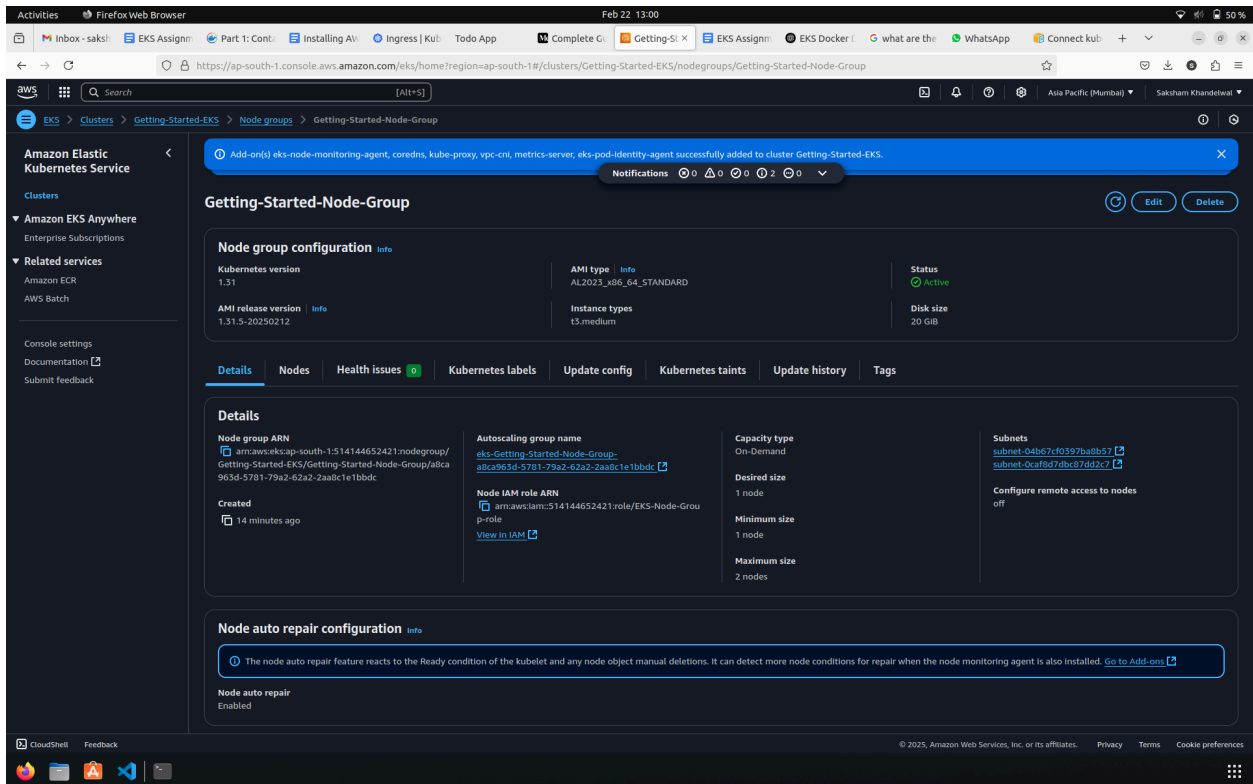
cloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Give the required add-ons



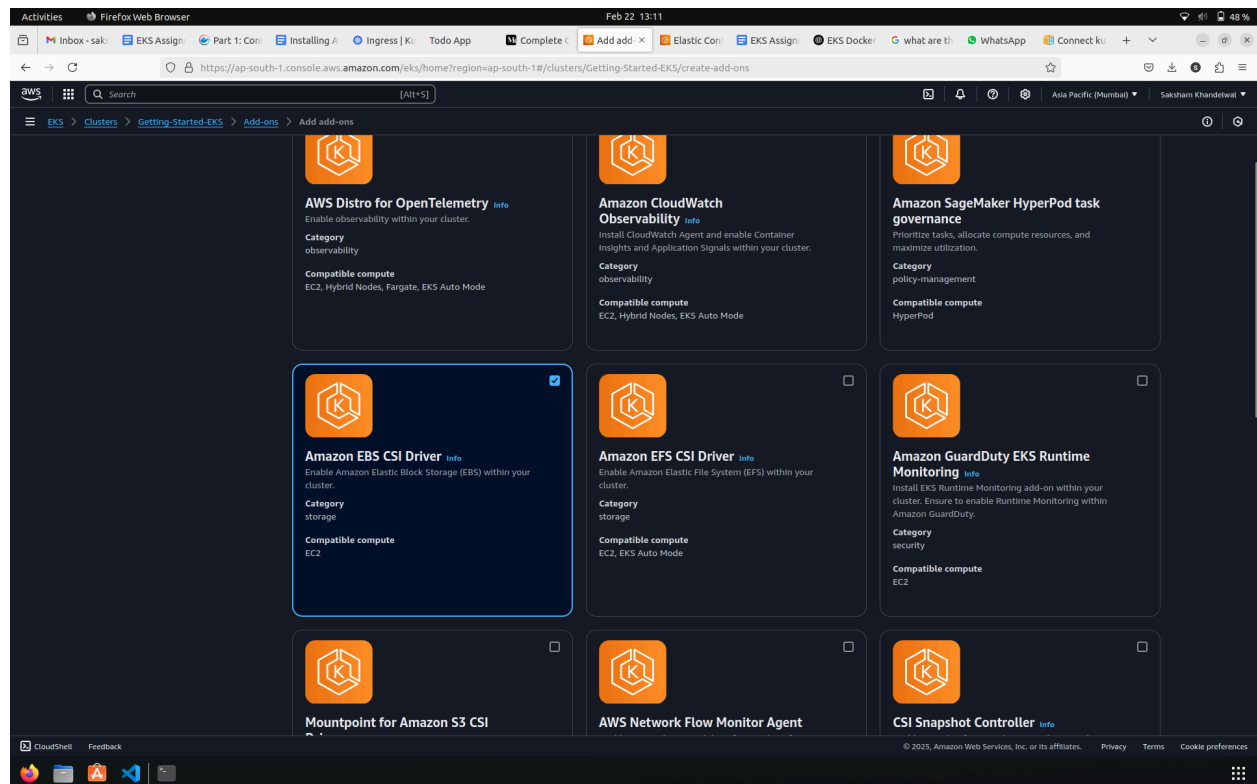
Step 6: Create node groups in the cluster.



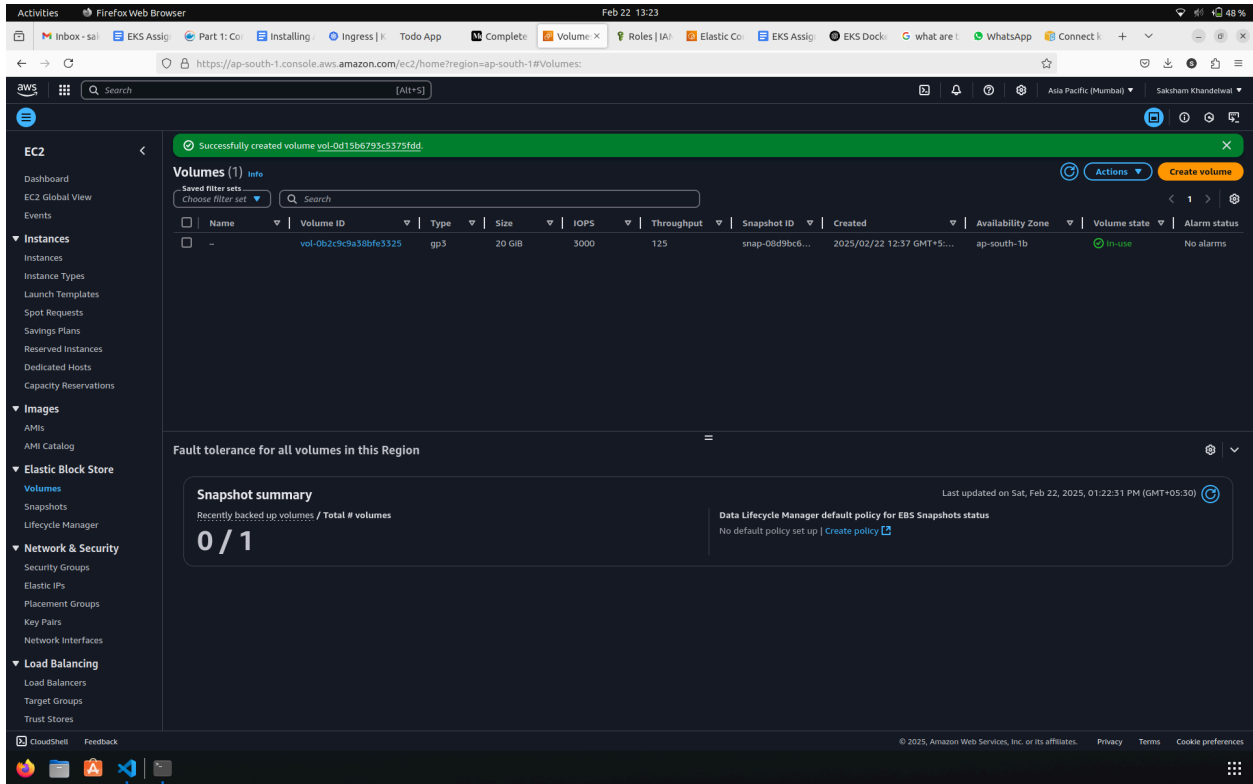
Step 7: Connect kubectl with EKS cluster.

```
saksham@Cker-sakshan:~/getting-started-app$ aws sts get-caller-identity
{
  "UserId": "514144652421",
  "Account": "514144652421",
  "Arn": "arn:aws:iam::514144652421:root"
}
saksham@Cker-sakshan:~/getting-started-app$ aws eks update-kubeconfig --region ap-south-1 --name Getting-Started-EKS
Added new context arn:aws:eks:ap-south-1:514144652421:cluster/Getting-Started-EKS to /home/saksham/.kube/config
saksham@Cker-sakshan:~/getting-started-app$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   172.20.0.1   <none>        443/TCP   33m
saksham@Cker-sakshan:~/getting-started-app$
```

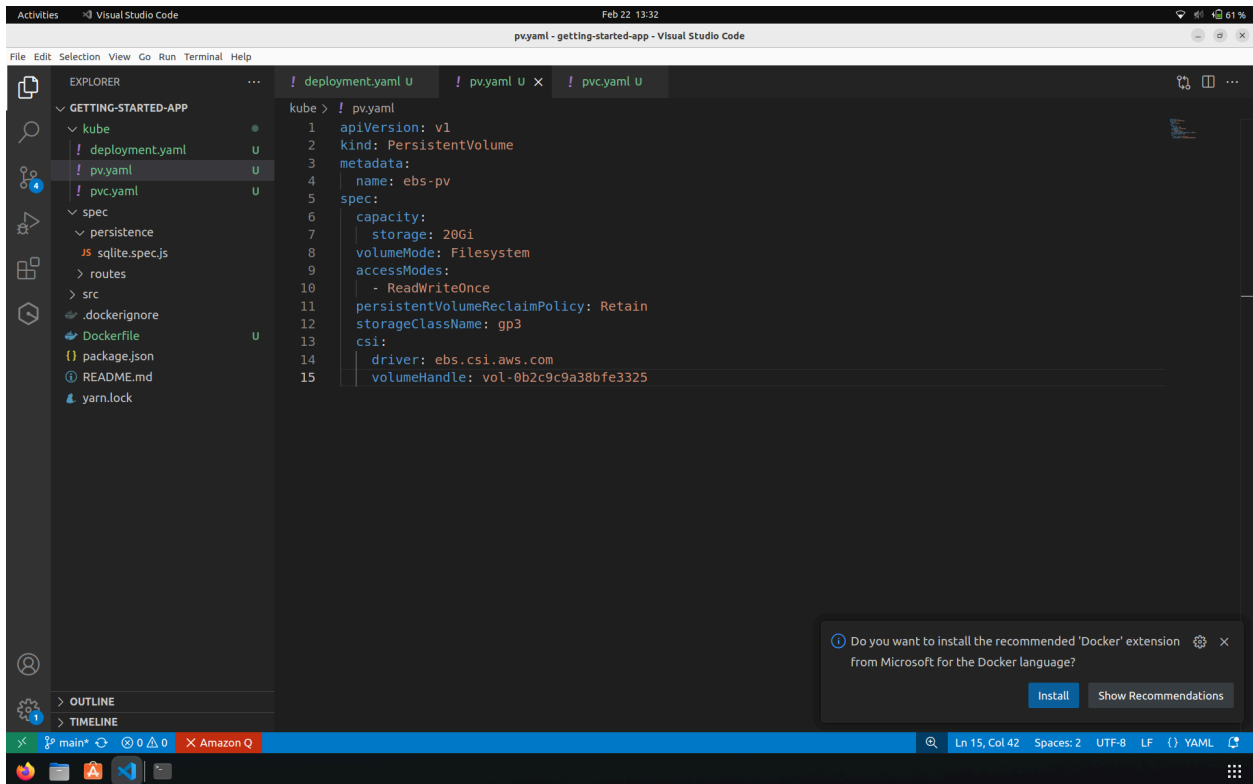
Step 8: For providing a persistent volume, a new add-on is needed.

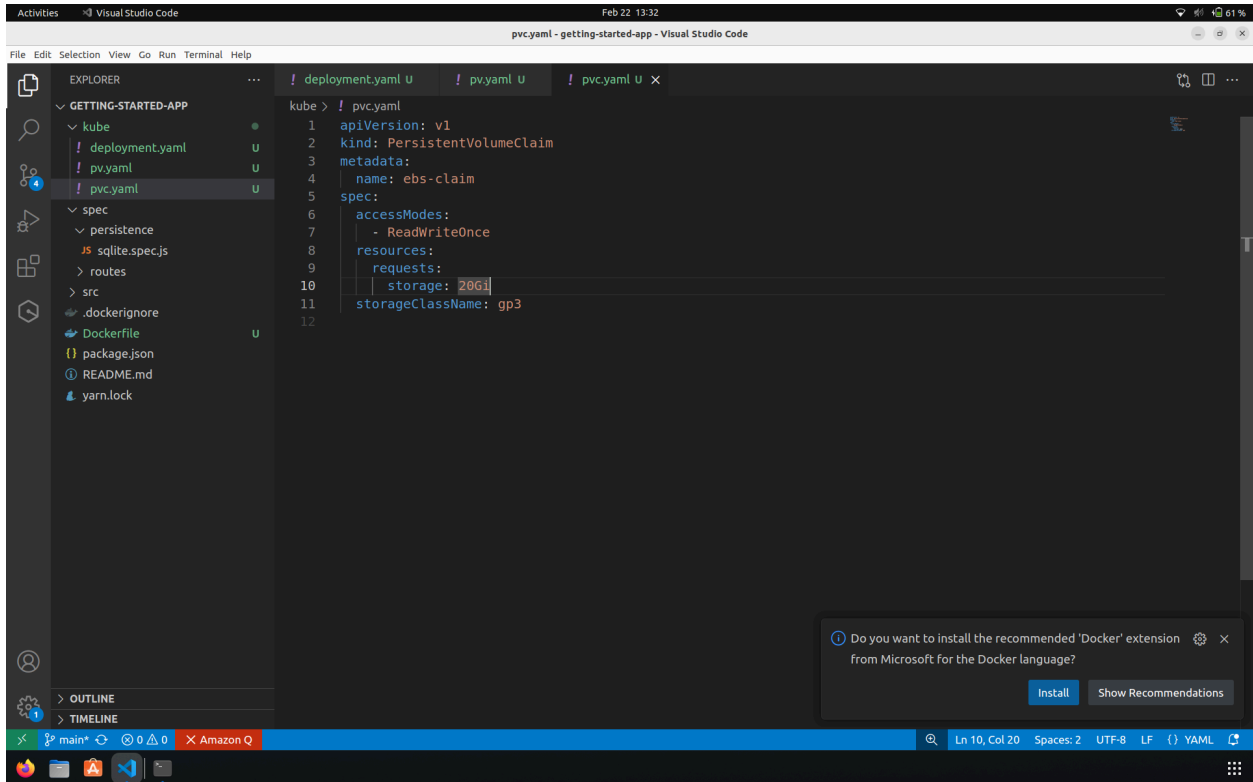


Step 9: Create a new EBS Volume.



Step 10: Create pv.yaml and pvc.yaml.





Step 11: Apply pv.yaml and pvc.yaml.

```

Added new context arn:aws:eks:ap-south-1:514144652421:cluster/Getting-Started-EKS to /home/saksham/.kube/config
saksham@Cker-saksham:~/getting-started-app$ kubectl get svc
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes ClusterIP  172.20.0.1   <none>        443/TCP    33m
saksham@Cker-saksham:~/getting-started-app$ cd kube/
saksham@Cker-saksham:~/getting-started-app/kube$ ls
deployment.yaml  pvc.yaml  pv.yaml
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl apply -f pv.yaml
persistentvolume/ebs-pv created
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl apply -f pvc.yaml
persistentvolumeclaim/ebs-claim created
saksham@Cker-saksham:~/getting-started-app/kube$

```

Step 12: Apply deployment.yaml.

```

saksham@Cker-saksham:~/getting-started-app/kube$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
getting-started-757bf7768b-f6k84    1/1     Running   0           7m40s
getting-started-757bf7768b-wzhhz    1/1     Running   0           7m40s
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl apply -f service.yaml

```

Step 13: Apply service.yaml

```

Normal Started      1s      kubelet      Started container getting-started
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
getting-started-757bf7768b-f6k84    1/1     Running   0           7m40s
getting-started-757bf7768b-wzhhz    1/1     Running   0           7m40s
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl apply -f service.yaml
service/getting-started-service created
saksham@Cker-saksham:~/getting-started-app/kube$ kubectl get services
NAME                                TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
getting-started-service             LoadBalancer 172.20.83.17  <pending>     80:30234/TCP 16s
kubernetes                           ClusterIP  172.20.0.1   <none>        443/TCP    121m
saksham@Cker-saksham:~/getting-started-app/kube$

```

Step 14: Give tags to subnets.

Resource configurations	Name	EKS-A2-2
Resource gateways	kubernetes.io...	owned
Target groups	kubernetes.io...	1
▼ DNS firewall	kubernetes.io...	owned

Step 15: Connect the eks cluster with an iam-oidc-provider

```
kubernetes ClusterIP 172.20.0.1 <none> 443/TCP 121m
saksham@Cker-saksham:~/getting-started-app/kube$ eksctl utils associate-iam-oidc-provider \
--region ap-south-1 \
--cluster Getting-Started-EKS \
--approve
2025-02-22 14:31:17 [i] will create IAM Open ID Connect provider for cluster "Getting-Started-EKS" in "ap-south-1"
2025-02-22 14:31:17 [✓] created IAM Open ID Connect provider for cluster "Getting-Started-EKS" in "ap-south-1"
saksham@Cker-saksham:~/getting-started-app/kube$
```

Step 16: Create IAM policy for load balancer controller

```
names are not allowed.
saksham@Cker-saksham:~/getting-started-app/kube$ aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy1 --policy-document file://iam-policy.json
{
  "Policy": {
    "PolicyName": "AWSLoadBalancerControllerIAMPolicy1",
    "PolicyId": "ANPAXPNLIZSCZLC4IH6Z2",
    "Arn": "arn:aws:iam::514144652421:policy/AWSLoadBalancerControllerIAMPolicy1",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 0,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2025-02-22T09:04:13+00:00",
    "UpdateDate": "2025-02-22T09:04:13+00:00"
  }
}
saksham@Cker-saksham:~/getting-started-app/kube$
```

Step 17: Create IAM service account.

```
saksham@Cker-saksham:~/getting-started-app/kube$ eksctl create iamserviceaccount \
--cluster=Getting-Started-EKS \
--namespace=kube-system \
--name=aws-load-balancer-controller \
--attach-policy-arn=arn:aws:iam::514144652421:policy/AWSLoadBalancerControllerIAMPolicy1 \
--override-existing-serviceaccounts \
--region ap-south-1 \
--approve
2025-02-22 14:36:28 [i] 1 iamserviceaccount (kube-system/aws-load-balancer-controller) was included (based on the include/exclude rules)
2025-02-22 14:36:28 [!] metadata of serviceaccounts that exist in Kubernetes will be updated, as --override-existing-serviceaccounts was set
2025-02-22 14:36:28 [i] 1 task: {
  2 sequential sub-tasks: {
    create IAM role for serviceaccount "kube-system/aws-load-balancer-controller",
    create serviceaccount "kube-system/aws-load-balancer-controller"
  }
}2025-02-22 14:36:28 [i] building iamserviceaccount stack "eksctl-Getting-Started-EKS-addon-iamserviceaccount-kube-system-aws-load-balancer-controller"
2025-02-22 14:36:28 [i] deploying stack "eksctl-Getting-Started-EKS-addon-iamserviceaccount-kube-system-aws-load-balancer-controller"
2025-02-22 14:36:28 [i] waiting for CloudFormation stack "eksctl-Getting-Started-EKS-addon-iamserviceaccount-kube-system-aws-load-balancer-controller"
2025-02-22 14:36:58 [i] waiting for CloudFormation stack "eksctl-Getting-Started-EKS-addon-iamserviceaccount-kube-system-aws-load-balancer-controller"
2025-02-22 14:36:58 [i] created serviceaccount "kube-system/aws-load-balancer-controller"
saksham@Cker-saksham:~/getting-started-app/kube$
```

Step 18: Helm install load balancer controller


```

Saving to: 'crds.yaml'

crds.yaml                                100%[=====] 29.70K  --.-KB/s   in 0.01s

2025-02-22 14:38:11 (2.12 MB/s) - 'crds.yaml' saved [30408/30408]

customresourcedefinition.apiextensions.k8s.io/ingressclassparams.elbv2.k8s.aws created
customresourcedefinition.apiextensions.k8s.io/targetgroupbindings.elbv2.k8s.aws created
saksham@Cker-saksham:~/getting-started-app/kube$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=G
etting-Started-EKS --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller --set region=ap-south-1 --set vpcId=vpc-04526f836b
333e600
NAME: aws-load-balancer-controller
LAST DEPLOYED: Sat Feb 22 14:39:57 2025
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
saksham@Cker-saksham:~/getting-started-app/kube$

```

Step 17: Apply ingress.yaml

```

EXPLORER
  GETTING-STARTED-APP
    kube
      crds.yaml
      deployment.yaml
      ingress.yaml
      pv.yaml
      pvc.yaml
      service.yaml
    src
      persistence
        index.js
        mysql.js
        sqlite.js
      routes
        additem.js
        deleteitem.js
        getitems.js
        updateitem.js
      static
        css
        js
          app.js
          babel.min.js
          react-bootstrap.js
          react-dom.production.min.js
          react.production.min.js
        index.html
        index.js
      OUTLINE
      TIMELINE

Ingress.yaml - getting-started-app - Visual Studio Code

kube > ! Ingress.yaml
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: ui-alb-1
5    annotations:
6      alb.ingress.kubernetes.io/target-type: ip
7      alb.ingress.kubernetes.io/scheme: internet-facing
8  spec:
9    ingressClassName: alb
10   rules:
11     - http:
12       paths:
13         - path: /
14           pathType: Prefix
15           backend:
16             service:
17               name: getting-started-service
18               port:
19                 number: 80

```

Step 18: Get the endpoint and browse it

